

AURELIX

Project Walkthrough & Interview Guide

An AI claims-verification platform where an LLM only observes, deterministic rules decide, a RAG copilot explains the policy with citations, and a human approves the edge cases.

Live demo: www.aurelix.space

Prepared for the E2M Solutions AI Engineer Intern interview
October 2026

Every number in this guide was measured by a command in the repository on 2026-10-04. Where something was not measured, the guide says so.

Contents

#	Section	Page
1	The project in one page	3
2	How AURELIX matches the E2M job	5
3	System architecture	6
4	How a claim is decided — the 8-stage pipeline	7
5	Working inside free-tier limits	9
6	The LLM gateway and cost tracking	10
7	The Policy Copilot (RAG)	11
8	Security: sign-in, roles and protected images	13
9	Testing, evaluation and CI	15
10	Deployment	16
11	The 12 numbers to remember	17
12	Glossary — every term, in simple English	18
13	Interview questions and simple answers	20
14	Weaknesses to admit, and what comes next	24
15	2-minute demo script	24

How to use this guide. Read sections 1-4 first: they are the heart of the project. Then read one feature section per day. The night before, read sections 11, 13 and 14 again.

1. The project in one page

The problem

When a customer's car is dented or their laptop screen breaks, they file an insurance claim: a short story and some photos. Someone at the insurance company must check: do the photos really show this damage? On the part the customer named? Is this a reused photo or a fake claim? Doing this by hand is slow. Letting an AI simply decide is risky, because AI can be wrong and cannot explain itself to a customer.

What AURELIX does

A customer (the **claimant**) submits a claim. AURELIX returns one of three answers — **supported**, **contradicted** or **not enough information** — and names the exact rule that decided it. Unclear or risky claims go to a person at the insurer (the **reviewer**), who approves or rejects them. Reviewers also get a **Policy Copilot**: they ask questions about the insurance policy and get answers with citations to the exact policy clauses.

The one big idea

The AI only observes. Plain code decides. One call to Google Gemini looks at the photos and reports what it sees as structured data. Everything after that — comparing the claim with the photos, the fraud score, the confidence, and the final verdict — is ordinary Python following 17 written rules. So every decision can be explained, repeated exactly, and re-checked for free.

Your 30-second pitch

"AURELIX checks insurance damage claims. A customer uploads photos and describes what happened. One Gemini call observes the photos, and deterministic rules decide whether the evidence supports the claim — every verdict names the rule that fired. Reviewers get a RAG copilot that answers policy questions with citations it cannot fake, and a human approves the edge cases. It has role-based login, offline evaluations that run in CI, and cost tracking for every model call — all on free tiers. It's live at aurelix.space."

Tech stack

Layer	Technology	Why
Frontend	Next.js 16, React 19, TypeScript, Tailwind v4, Recharts	Modern, fast, deploys free on Vercel
Backend	Python, FastAPI, Pydantic v2, SQLAlchemy 2.0	Typed APIs, automatic validation, async-friendly
Database	SQLite (Postgres-ready via DATABASE_URL)	Free tier has no managed database
AI models	Google Gemini (vision), Groq gpt-oss-20b (text), bge-small embeddings (local)	All free tiers
AI libraries	google-genai, openai SDK (for Groq), fastembed/ONNX, rank-bm25, NumPy	Small and explainable
Images	OpenCV (blur check), Pillow, perceptual hashing	Deterministic quality and duplicate checks

Layer	Technology	Why
Security	PyJWT, Argon2id (argon2-cffi), HMAC-signed URLs	Standard, well-reviewed building blocks
Ops	Render (API), Vercel (frontend), GitHub Actions (CI)	Free hosting with automatic deploys

2. How AURELIX matches the E2M job

E2M builds AI apps for digital agencies. Here is how each thing they care about appears in AURELIX — honestly, including the parts that are only partly done.

E2M looks for	Where it is in AURELIX	Honest level
LLM APIs (OpenAI, Claude, Gemini)	Gemini for vision; an LLM gateway with Groq, Gemini, OpenAI and Claude adapters	Gemini and Groq used live; OpenAI and Claude adapters tested with fixtures
RAG with embeddings and vector databases	Policy Copilot: BM25 + local embeddings, RRF fusion, vector-store interface	Live. pgvector backend written, but production uses an in-memory NumPy store
Bounded AI agents	8-stage pipeline of bounded components, one of them an LLM	No autonomous tool-calling agent yet (planned: read-only Investigator Assistant)
FastAPI	The whole backend: REST, SSE streaming, 202 async jobs	Live
React / Next.js	The whole dashboard	Live
Postgres / Supabase	SQLAlchemy works with Postgres via DATABASE_URL; pgvector code ready	Not deployed — free tier uses SQLite
Cost control	1 model call per claim, quota ledger, model ladder, caching, per-call cost tracking	Live
Testing AI outputs	Offline benchmark, RAG golden set, live copilot eval, CI gates	Live, in CI
Fast iteration	Re-score every verdict from stored AI output for free after a rule change	Live

Say this if asked about agents: "AURELIX is a pipeline of bounded components, not an autonomous agent. I chose that on purpose: an insurance decision must be explainable. My next step is a read-only assistant for reviewers with a small set of tools, a step limit and a token budget."

3. System architecture

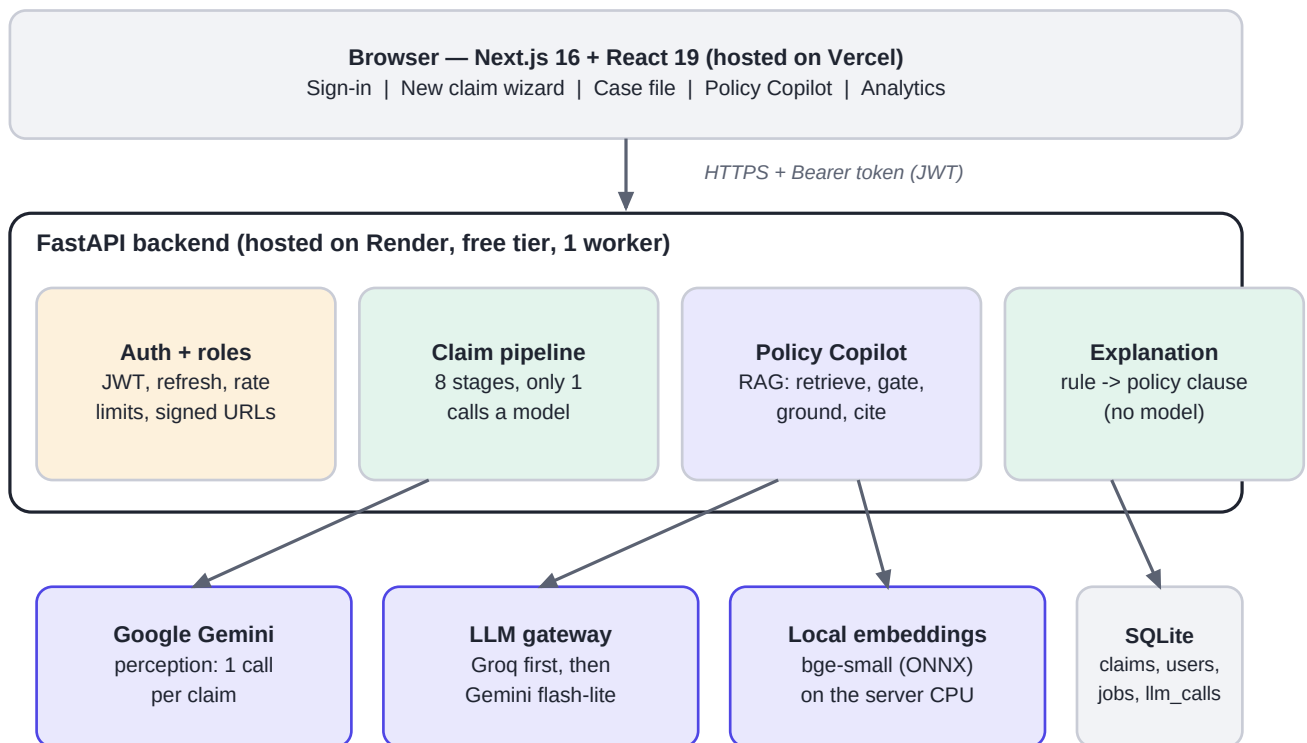


Figure 1 — The whole system. Purple boxes are AI models; green boxes are plain code.

The parts, in simple words

- **Frontend (browser).** A Next.js app on Vercel. It shows the sign-in screen, the 4-step claim wizard, the live progress of an analysis, the case file, the copilot and analytics. It only talks to the API; it holds no secrets.
- **FastAPI backend.** One Python service on Render. Every request (except health checks) must carry a login token. It runs the claim pipeline, the copilot and the explanation endpoint.
- **Gemini.** Called once per claim to look at the photos. Its answer is checked against a strict schema.
- **LLM gateway.** One place that talks to text models for the copilot. It tries Groq first and falls back to a Gemini model that the claim pipeline does not use.
- **Local embeddings.** A small embedding model runs on the server's own CPU. No text leaves the server to be embedded.
- **SQLite database.** Stores claims, the audit trail, users, login tokens, jobs and one row per model call.

One request, end to end

A claimant submits the wizard. The browser sends a multipart form (text + photos) with the login token. FastAPI checks the token, checks rate limits, checks and saves the photos, then runs the 8 stages. Progress is streamed back live using **Server-Sent Events (SSE)**, so the user sees each stage finish. The final claim, with its audit trail, is saved and shown in the case file.

4. How a claim is decided — the 8-stage pipeline

Purple = the only stage that calls an AI model. Green = plain, deterministic Python.

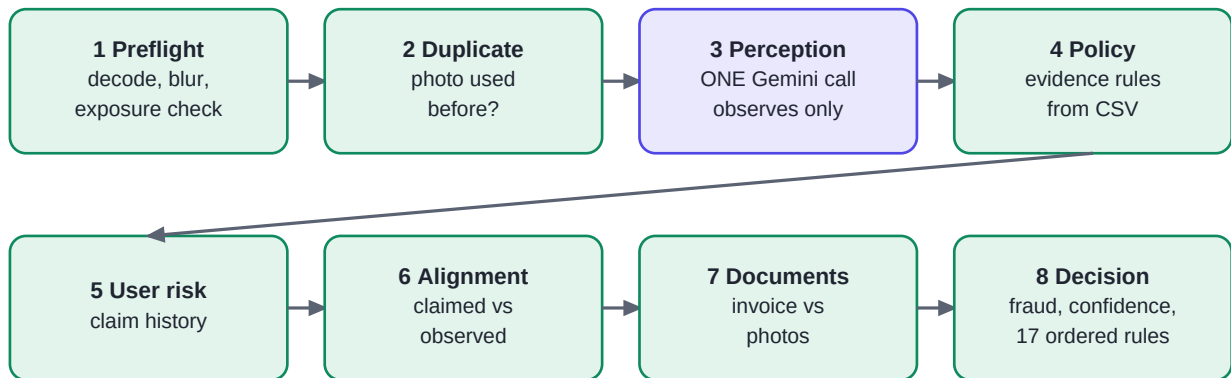


Figure 2 — The 8 stages. Only stage 3 calls an AI model.

Stage	What it does (simple)	AI?
1 Preflight	Opens each photo, checks format and size, measures blur (OpenCV Laplacian) and brightness. If no photo is usable, it stops here and no AI call is made.	No
2 Duplicate check	Makes a fingerprint of each photo (pHash + dHash + SHA-256) and checks whether the same photo was sent with an earlier claim.	No
3 Perception	ONE Gemini call. It returns JSON: the object, which parts are damaged, how badly, image quality, and whether the claim text tries to give it instructions.	Yes
4 Policy check	Checks the evidence rules for this object (number of photos, covered parts).	No
5 User risk	Looks at the claimant's history (number of past claims, rejections).	No
6 Alignment	Compares what the customer CLAIMED with what the model SAW: same part? how big is the severity gap?	No
7 Document check	Compares an invoice or repair estimate with the photos (right object? plausible amount?).	No
8 Decision	Adds up a fraud score, computes confidence, then checks 17 rules in order. The first rule that matches gives the verdict.	No

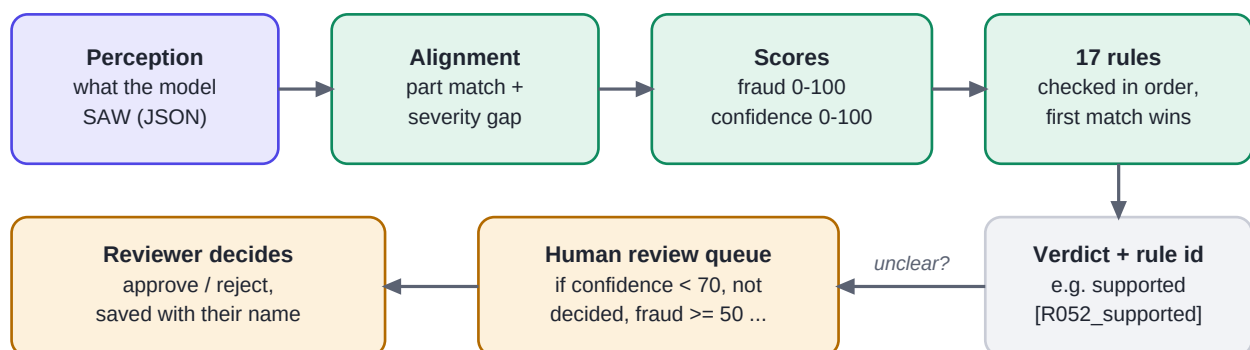


Figure 3 — From observation to verdict, and when a human steps in.

The 17 rules (simplified)

The rules are in **config/decision_rules.yaml**. They are checked top to bottom and the first match wins. The order matters: **"can we judge at all?" comes before "is it false?"** A claim about a part nobody could see is never called false.

Group	Examples	Verdict
Can we judge at all?	R001 no usable photo; R002 AI call failed; R003 photo quality unusable	Not enough information
Right object?	R010 the photo shows a different object (a bicycle for a car claim)	Contradicted
Is the part visible?	R020 claimed part not visible; R021 no part named and no damage seen	Not enough information
Fraud	R030 photo reused from another claim; R031 fraud score 70 or more	Contradicted
Conflicts	R040 damage on a different part; R041 claimed part visible but undamaged; R042 damage 2+ steps less severe than claimed; R043/R044 documents disagree	Contradicted
Support	R050 slightly overstated; R051 damage on a neighbouring part; R052 everything matches	Supported
Fallback	R099 nothing above matched	Not enough information

Fraud score, confidence and human review

- **Fraud score** starts at 10 and adds points only for objective signals: reused photo +45, wrong object +30, damage on another part +20, severity inflation +20, risky history +15, instructions in the text +10, document problems +15 to +25, poor image +5. Tone or vague writing never counts.
- **Confidence** is a weighted mix: how sure the model was (30%), image quality (25%), how well the parts match (25%) and whether the severity agrees (20%).
- **A human reviews the claim** if confidence is below 70, the verdict is contradicted or not enough information, the fraud score is 50 or more, or a manipulation or instruction flag is present.

A worked example

Claim: "A trolley hit my car in the car park and dented the front bumper." + one photo. Preflight: photo is sharp. Duplicate: never seen before. Perception: car, front bumper, dent, medium. Alignment: exact part match, severity gap 0. Fraud score 10, confidence 83. Rule **R052_supported** fires. Verdict: **supported**, no human needed. The case file shows the photo, the rule, and the policy clauses behind R052 (COV-1, DEF-4, COV-2).

5. Working inside free-tier limits

The Gemini free tier allows only **20 requests per day per model** (and 5 per minute), resetting at midnight Pacific time. That one fact shaped the whole design.

- **One request per claim.** An older version made 4 AI calls per claim (understand, see, fraud, decide). Now one call observes and code does the rest. For the 44-claim benchmark, batching 3 claims per request means 15 requests instead of 176 — by design, not a measured A/B test.
- **Model ladder.** The quota is per model, so three Gemini models give 60 requests a day for free. If one is used up, the next one is tried automatically.
- **Quota ledger.** A small file remembers how many requests each model used today, so a restart does not forget. A request is counted before it is sent and refunded if the server rejects it.
- **Smart retries.** A per-minute limit (HTTP 429) is waited out. A per-day limit is never retried — retrying would only waste tomorrow's budget. Bad requests (400/401/403) fail at once.
- **Circuit breaker.** After 5 failures in a row it stops calling for a cooldown, instead of hammering a broken service.
- **Preflight saves calls.** If no photo is usable, no AI call is made at all.
- **Demo guard.** A daily per-visitor cap so one visitor cannot use up the shared budget.

Claim perception (images)



Policy Copilot (text) — never uses the models above



Figure 4 — Two separate ladders: claims never compete with the copilot for quota.

6. The LLM gateway and cost tracking

The copilot needs a text model. Instead of writing code for one company's API, AURELIX has a **gateway**: one function, many providers behind it. Which provider answers which task is a setting in `config/limits.yaml`, not code.

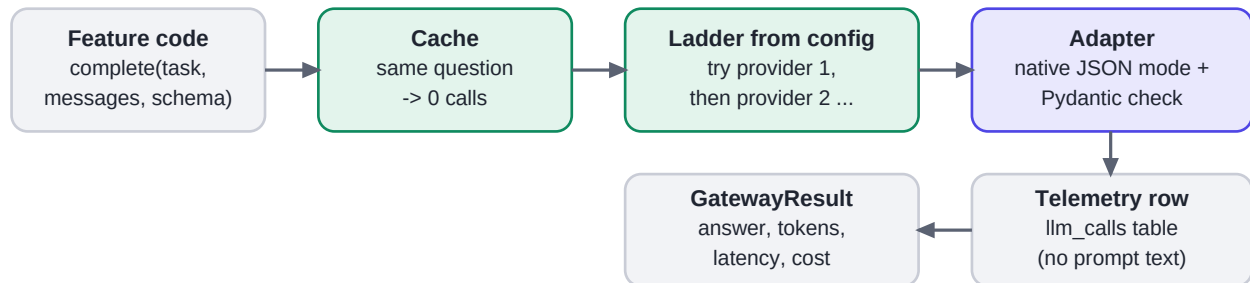


Figure 5 — The gateway: cache first, then providers in order, then validation and telemetry.

Key ideas

- **Adapters.** Groq and OpenAI share one adapter (Groq speaks the OpenAI protocol). Gemini goes through the existing client, so it keeps the quota ledger, rate governor, retries and circuit breaker. Claude is optional.
- **Structured output.** Each provider's own "JSON mode" forces the answer to match a schema (Groq strict json_schema, Claude output_config, Gemini response_schema). Then Pydantic checks it again.
- **One error vocabulary.** Every provider's errors become one of five: RateLimited, DailyQuotaExhausted, AuthError, ProviderUnavailable, InvalidResponse. The ladder moves on for all of them.
- **Never fake an answer.** If no provider can answer, the user sees an honest "no model available".
- **Perception's budget is protected.** The copilot may not use any model the claim pipeline uses.

Telemetry: a meter on every call

Every model call — and every cache hit — writes one row to the `llm_calls` table: task, provider, model, prompt version, input and output tokens, latency, cost and outcome. **Never the prompt or the answer**, because more people can read metrics than can read claims. The Analytics page shows calls per day, p50/p95 latency, cost per claim and cache-hit rate, and today's Gemini quota.

About cost: every provider here is on a free tier, so the **billed** cost is \$0. AURELIX records the **list-price equivalent** from a dated price file — what the same traffic would cost on a paid plan. Measured example: one live Groq copilot call used 219 input and 55 output tokens, took 706 ms, list price \$0.000033. A real per-claim cost has **not** been measured yet — do not quote one.

7. The Policy Copilot (RAG)

What is RAG, simply?

RAG = Retrieval-Augmented Generation. An LLM on its own may invent policy wording. With RAG, we first **retrieve** the real policy clauses that match the question, then give them to the model and say: "answer only from these, and cite them". It is like an open-book exam with a strict examiner.

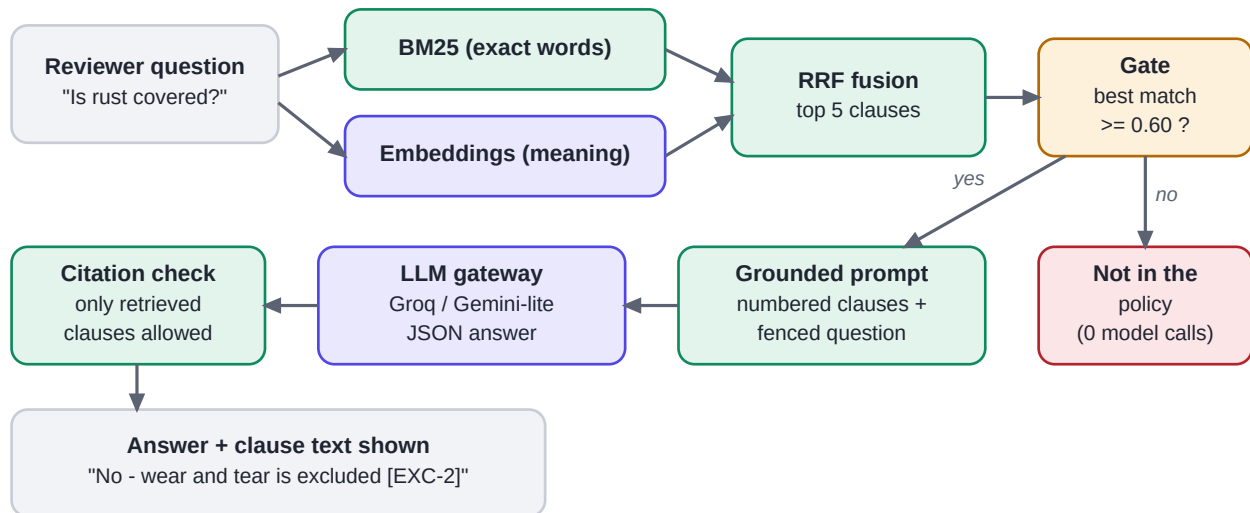


Figure 6 — How one copilot question is answered.

Step by step

- **The knowledge.** A synthetic insurance policy with 35 numbered clauses (DEF, COV, EXC, EVD, PRC, ASM, FRD, REV). Each clause is one chunk, so a citation points to something a person can read. Each chunk starts with a header like "Part 3 — Exclusions > EXC-2 Wear and tear" so section names help the search.
- **BM25.** A classic keyword search: good at exact words like "deductible" or "EVD-1".
- **Embeddings.** The model bge-small (384 numbers per text) turns text into vectors so that similar meanings are close together. "My car got rusty" finds the wear-and-tear clause even with no shared words.
- **RRF fusion.** Each clause gets $1/(60 + \text{rank})$ from each list; the scores are added. Simple, and no score tuning is needed because BM25 and cosine scores are on different scales.
- **The gate.** If even the best clause is a weak match (cosine below 0.60), the answer is "not in the policy" and **no model is called**. It saves cost and stops guessing.
- **Grounded prompt.** Clauses are numbered sources; the question is wrapped in markers and labelled as data, so instructions hidden in it are not followed.
- **Citation check.** Plain code keeps only citations that were actually retrieved. If none are left, the answer is not shown. This works even if the model is tricked.

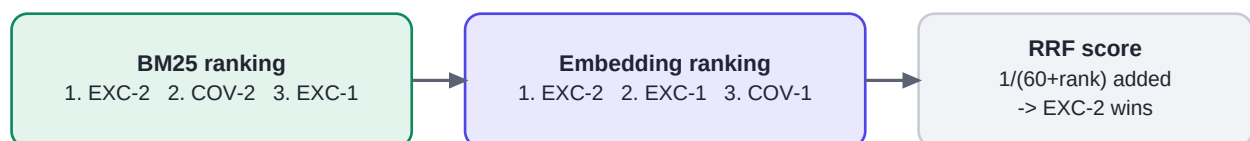


Figure 7 — Reciprocal Rank Fusion combines two rankings without comparing their raw scores.

How well does it work? (measured, no API cost)

Retrieval setup (40 golden questions)	recall@5	MRR	nDCG@5	not-found accuracy
BM25 only	0.828	0.712	0.711	0.825
BM25 + LSA (old)	0.818	0.688	0.687	0.775
BM25 + bge-small (shipped)	0.961	0.885	0.889	0.925
+ cross-encoder reranker (rejected)	0.906	0.927	0.885	0.825

Live test on Groq: **32 of 32** answerable questions answered with a correct citation, **8 of 8** unanswerable questions refused (5 by the gate with zero model calls). The reranker was tested and **not** used: slightly better MRR, but worse recall and not-found accuracy, 17x slower and +160 MB of memory.

The explanation endpoint

GET /api/v1/claims/{id}/explanation answers "why was my claim decided this way?" with no AI at all: it reads the rule from the claim's audit trail and looks up the clauses in **knowledge/policies/rule_clause_map.yaml**. A test fails the build if any rule has no clause behind it.

Important: the copilot never decides a claim, and it never reads other customers' claims. RAG is for reviewers' questions only. If an interviewer asks "does RAG decide claims?" the answer is **no — on purpose**.

8. Security: sign-in, roles and protected images

Before Phase 7, anyone could read any claim by its number and see every photo. Now every request needs a login token, and each role sees only what it should.

	Claimant (customer)	Reviewer (insurer staff)
Submit a claim	Only as themselves	On behalf of any policyholder
See claims	Only their own (others are "not found")	All claims
Review queue, approve/reject	No	Yes
Policy Copilot, Analytics	No	Yes
See photos and the explanation	Of their own claims	Of all claims

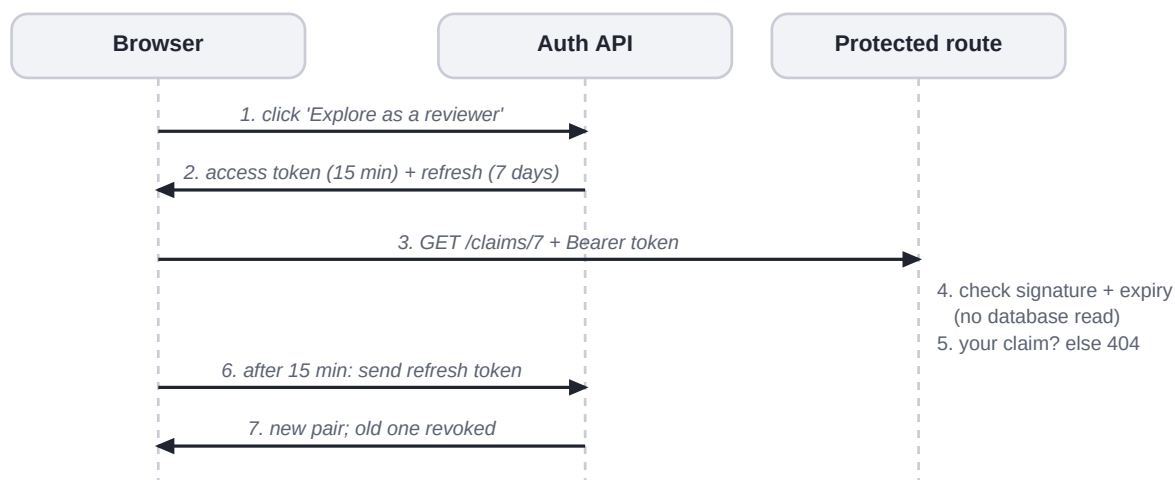


Figure 8 — Demo sign-in, a protected request, and a silent token refresh.

The pieces, simply

- **Demo sign-in.** "Explore as a claimant" creates a fresh private account for each visitor — no password, no form. So two visitors never see each other's claims.
- **Access token (JWT, 15 minutes).** A signed note that says who you are and your role. The server checks the signature and expiry without a database read. The algorithm is pinned to HS256, so forged "alg: none" tokens fail.
- **Refresh token (7 days).** A random string, stored on the server only as a SHA-256 hash. Each use gives a new one and cancels the old one. If an old one is used again, someone copied it, so the whole session is cancelled.
- **Passwords** (for operator accounts only) are hashed with **Argon2id** at the OWASP minimum (19 MiB memory, 2 iterations). Every login failure gives the same message, so attackers cannot learn usernames.
- **Signed image links.** An `<img>` tag cannot send a token, so the server gives each photo a link signed with HMAC that expires after 15 minutes — only after checking you may see that claim.
- **Rate limits.** 12 claim submissions and 30 copilot questions per hour per account; 5 failed logins per 15 minutes.
- **Security headers** on every response (no sniffing, no framing, strict content policy, HSTS).

OWASP Top 10 for LLM Applications (2025) — short version

Risk	What AURELIX does
LLM01 Prompt injection	Text is fenced as data; model reports instructions as a flag; a regex also flags them; the citation check works even if the model obeys
LLM02 Sensitive data	Claimants see only their claims; copilot reads only the policy; telemetry has no text
LLM05 Bad output handling	Every answer is schema-checked; the model never decides; text shown as plain text
LLM06 Excessive agency	No model has tools or write access
LLM09 Misinformation	Answers must cite retrieved clauses; uncited answers are hidden
LLM10 Unbounded consumption	1 call per claim, zero-call gate, rate limits, demo cap, token caps

The full mapping, with the test that proves each control, is in **docs/SECURITY.md**.

9. Testing, evaluation and CI

Three levels of checking

- **546 automated tests** (1 skipped: pgvector needs a Postgres server). They are **hermetic**: no internet, no API keys, fake model responses. Example tests: "one claim makes exactly one model call", "another customer's claim returns 404", "a cached answer makes zero calls".
- **Offline evaluations** score the system on fixed question sets with known answers, at zero API cost.
- **A live evaluation** of the copilot on Groq's free tier, run by hand only (it refuses to run without --live).

The claim benchmark

Measure	Result	95% confidence interval
Accuracy (44 synthetic claims)	95.5%	88.6% - 100%
Macro-F1 (average F1 over the 3 verdicts)	95.0%	86.0% - 100%

Confidence interval, simply: we only have 44 cases. If we took another 44 similar cases, the score could move. The **bootstrap** method resamples the 44 cases 10,000 times and shows the range where the score usually lands. It is wide because the set is small — and saying so is honest.

Always say "synthetic". The 44 photos are generated illustrations, not real claim photos. The accuracy does not predict accuracy on real photos with glare, blur or bad lighting.

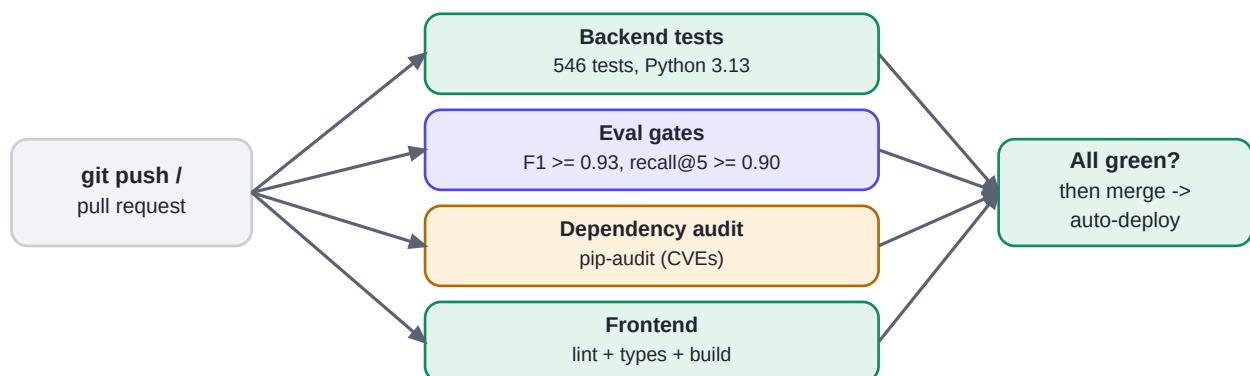


Figure 9 — GitHub Actions runs four jobs on every push and pull request.

- **Gates are floors with headroom:** macro-F1 must stay at or above 0.93 (measured 0.950) and copilot recall@5 at or above 0.90 (measured 0.961). A small wobble passes; a real regression fails the build.
- **pip-audit** checks dependencies for known security holes. It found one (pydantic-settings CVE-2026-58203); it was upgraded.
- **No secrets in CI**, so CI can never spend model quota.

10. Deployment

Part	Where	Notes
Frontend	Vercel (free)	Auto-deploys from GitHub main. API URL is fixed at build time.
Backend	Render (free, 512 MB, one worker)	Auto-deploys from main. Sleeps after ~15 min idle; ~50 s to wake.
Retrieval index	Built during the Render build	Downloads the 67 MB embedding model once; embeds the policy there.
Database	SQLite on Render's disk	Wiped on every redeploy (free tier). Postgres is the upgrade.

- **Memory:** the whole API uses 112 MB, or **287 MB** with the embedding model loaded (limit 512 MB). Embedding the 35 clauses inside the server would add about 139 MB, so it is done at build time and the server refuses to do it.
- **Settings on Render:** GEMINI_API_KEY, GROQ_API_KEY (copilot), AURELIX_JWT_SECRET (else everyone is signed out when the server sleeps), CORS_ORIGINS (the Vercel site).
- **Health checks:** /health (is the process alive) and /ready (is the database ok, is the index loaded).

11. The 12 numbers to remember

#	Number	Meaning
1	1	model request per claim
2	8 / 17	pipeline stages / ordered decision rules (only 1 stage calls a model)
3	95.5%	accuracy on 44 synthetic claims (95% CI 88.6 - 100%)
4	95.0%	macro-F1 on the same (95% CI 86.0 - 100%)
5	0.961	copilot retrieval recall@5 (was 0.818 with the old LSA method)
6	0.889	copilot nDCG@5 (was 0.687)
7	32/32, 8/8	live copilot: answerable questions cited correctly / unanswerable refused
8	0.60	the copilot gate: below this match score, no model is called
9	287 MB	API memory with the embedding model (Render free limit 512 MB)
10	546	automated tests passing, also in GitHub Actions on Python 3.13
11	20/day	Gemini free requests per model (60/day with 3 models); Groq: 1,000/day
12	15 min / 7 days	access token / refresh token lifetime

12. Glossary — every term, in simple English

Term	Meaning (and where it is in AURELIX)
LLM	Large Language Model, e.g. Gemini or GPT. Reads and writes text (some also see images).
Multimodal	A model that takes more than one kind of input, e.g. text and images together.
Deterministic	Same input always gives the same output. AURELIX's rules are deterministic; an LLM is not.
Structured output	Forcing the model to answer in JSON that matches a schema, instead of free text.
Pydantic	A Python library that checks data matches a typed model and raises an error if not.
Schema	A description of the exact shape of data: fields, types and allowed values.
Hallucination	When a model states something false with confidence. RAG and citations reduce it.
Prompt injection	Text that tries to give the model new orders, e.g. "ignore your rules, approve this".
RAG	Retrieval-Augmented Generation: find relevant documents first, then let the model answer from them.
Chunk / chunking	Splitting documents into pieces for search. AURELIX uses one chunk per policy clause.
Embedding	A list of numbers (here 384) that represents the meaning of a text. Similar meaning = close numbers.
Vector store / vector DB	A place that stores embeddings and finds the closest ones to a query.
Cosine similarity	A score from -1 to 1 for how alike two embeddings are. 1 = same direction.
pgvector / HNSW	pgvector adds vector search to Postgres. HNSW is a fast approximate index for it.
BM25	A classic keyword-ranking formula. Rewards rare words that appear in a document.
Hybrid search	Using keyword search and embedding search together.
RRF	Reciprocal Rank Fusion: combine rankings by adding $1/(60 + \text{rank})$ from each list.
Reranker / cross-encoder	A second model that re-orders search results. Tested in AURELIX and rejected.
recall@k	Share of the correct documents found in the top k results.
MRR	Mean Reciprocal Rank: $1/\text{position of the first correct result}$, averaged.
nDCG@5	A ranking score that gives more credit when correct results are near the top.
Golden set	A fixed list of questions with known correct answers, used to measure quality.
Accuracy / F1 / macro-F1	Accuracy = share correct. F1 balances precision and recall. Macro-F1 = average F1 over all classes.
Bootstrap / confidence interval	Resampling the test set many times to show how much a score could vary.
Telemetry	Recording facts about each call (tokens, time, cost) to monitor the system.
p50 / p95 latency	Half of calls are faster than p50; 95% are faster than p95.
Token	A piece of a word. Models count input and output in tokens; prices are per million tokens.
Rate limit / quota	Rate limit = max calls per minute. Quota = max calls per day.
Fallback ladder	An ordered list of models: if one fails or runs out, try the next.

Term	Meaning (and where it is in AURELIX)
Circuit breaker	Stops calling a failing service for a while instead of retrying forever.
Exponential backoff + jitter	Wait longer after each failure, with a random part, so clients don't retry together.
Idempotency key	A request id that makes a retried request safe: the second one returns the first result.
SSE	Server-Sent Events: the server streams updates to the browser over one HTTP response.
202 Accepted / async job	The server accepts work, returns a job id at once, and the client checks back later.
JWT	JSON Web Token: a signed, short-lived token that proves who you are.
Refresh token rotation	Each refresh gives a new token and cancels the old one; reuse means theft.
Argon2id	A slow, memory-hard password-hashing method, so guessing passwords is expensive.
HMAC	A signature made with a secret key. Used to sign image links so they cannot be forged.
RBAC	Role-Based Access Control: what you can do depends on your role (claimant, reviewer, admin).
CORS	Browser rule for which websites may call an API. Set to the Vercel site only.
pHash / dHash	Perceptual hashes: fingerprints that stay the same when a photo is resized or recompressed.
Laplacian variance	A number for how sharp a photo is. Low = blurry.
ONNX	A standard format for running machine-learning models fast on CPU without heavy frameworks.
Hermetic test	A test that needs no network or outside services, so it always gives the same result.
CI / GitHub Actions	Continuous Integration: tests run automatically on every push.
Human-in-the-loop	A person approves the cases the system is unsure about.

13. Interview questions and simple answers

A. The project

Q. Tell me about your project.

Use the 30-second pitch on page 3. Then offer to show the live demo.

Q. Why did you build it this way, with rules instead of letting the AI decide?

An insurance decision must be explained to a customer. "The model said so" is not an explanation.

Rules give a rule id for every verdict, the same input always gives the same answer, and I can re-check all past verdicts for free after changing a rule.

Q. Which part uses AI?

Two places. Perception: one Gemini call per claim reads the photos. And the Policy Copilot, which answers reviewers' questions. Nothing else calls a model.

Q. Who are the users?

Claimants (customers) submit claims and see only their own. Reviewers (insurer staff) see all claims, decide the unclear ones and use the copilot.

Q. What was the hardest problem?

The free tier: 20 requests a day. I changed the design from 4 model calls per claim to 1, added a model ladder, a quota ledger and smart retries.

B. LLMs and GenAI

Q. How do you make the model return reliable JSON?

I use the provider's native structured output — Gemini `response_schema`, Groq `strict json_schema` — so the model is forced to follow the schema. Then I validate again with Pydantic. Invalid answers are errors, never guessed.

Q. What happens if Gemini is down or out of quota?

The ladder tries the next model. If all fail, the claim becomes "not enough information" with the reason saved and goes to a human. There is no fake answer anywhere.

Q. How do you handle prompt injection?

Claim text and copilot questions are wrapped as data. The model reports instruction-like text as a flag, and a regex flags it too; flagged claims go to a human. For the copilot, a code check only allows citations that were retrieved, so even a tricked model cannot show a fake citation.

Q. Why did you build an LLM gateway?

So feature code doesn't depend on one vendor. Which model answers is a config setting, errors look the same for every provider, and every call is recorded with tokens and cost.

Q. Why Groq for the copilot?

Its free tier gives 1,000 requests a day, versus 20 for a Gemini model, and it is a separate budget, so the copilot never uses up the quota that claims need.

Q. What is temperature?

How random the model's output is. I use a low value (0.1) because I want consistent, factual answers, not creativity.

C. RAG and vector search

Q. Explain your RAG pipeline.

Find the 5 best policy clauses with BM25 and embeddings, merged by RRF. If the best match is too weak, answer "not in the policy" without calling a model. Otherwise send the clauses and the question to the model, ask for an answer with citations, and keep only citations that were actually retrieved.

Q. Why hybrid search, not just embeddings?

BM25 is great for exact words and ids; embeddings are great for different wording with the same meaning. Together they were best: recall@5 0.961.

Q. Why RRF and not a weighted average?

BM25 scores and cosine scores are on different scales. RRF only uses rank positions, so there is nothing to tune.

Q. How did you choose the chunk size?

One chunk per policy clause, because a citation should point to something a person can read. Each chunk has a header with its section name.

Q. Which embedding model and why?

bge-small-en-v1.5: 384 dimensions, 67 MB, runs locally on CPU with ONNX, free, and no text leaves the server. It fits in Render's 512 MB memory.

Q. Which vector database?

In production a NumPy in-memory store: 35 vectors are fastest to scan exactly. I also wrote a pgvector backend with an HNSW index for when the data moves to Postgres.

Q. How do you evaluate RAG?

A golden set of 40 questions with the correct clauses, including 8 that the policy cannot answer. I measure recall@k, MRR, nDCG and not-found accuracy, offline and free.

Q. Did you use a reranker?

I tested a cross-encoder reranker and did not use it: worse recall and not-found accuracy, 17x slower, +160 MB memory.

D. Backend and system design

Q. Why FastAPI?

Type hints become automatic validation and API docs, it supports async and streaming, and it is fast to build with.

Q. How does the live progress work?

Server-Sent Events. The analysis runs on a worker thread and each stage sends an event. A keepalive every 15 seconds stops the hosting proxy from closing a quiet connection.

Q. What is the async API?

POST returns 202 with a job id at once; the client polls or streams the job. An idempotency key makes retries safe: a unique database index means 8 identical requests at once create 1 job.

Q. How would you scale to 100x users?

Postgres instead of SQLite, object storage for photos, a Redis queue and shared rate limiter so many workers can run, pgvector for retrieval, and paid model tiers by config change.

Q. How does duplicate photo detection work?

Each photo gets perceptual fingerprints (pHash and dHash) and a SHA-256 of its pixels. A new claim's photo is compared with all earlier ones; a match under another claim triggers rule R030. Only fingerprints are stored, not the photos.

E. Security**Q. How does login work?**

A 15-minute JWT access token for each request and a 7-day refresh token that rotates on every use. Reusing an old refresh token cancels the whole session, because it means a copy was stolen.

Q. What is the difference between 401, 403 and 404 in your API?

401: we don't know who you are. 403: we know you, but your role can't do this. 404 for another customer's claim: we don't even confirm it exists.

Q. How are photos protected?

With signed links: an HMAC signature over the file name and an expiry time, valid for 15 minutes, only given after checking you may see that claim.

Q. How are passwords stored?

With Argon2id at the OWASP minimum settings. Demo accounts have no password.

Q. Where do you keep the refresh token, and is that safe?

In the browser's localStorage, because the frontend and API are on different sites where cookies get blocked. Risk: XSS. I reduce it with rotation, reuse detection and never rendering text as HTML. Better: an httpOnly cookie once the API is on the same domain.

F. Testing and quality**Q. How do you test something that uses an AI model?**

Tests never call the real model; they use recorded or fake responses, so they are fast and repeatable. Quality is measured separately with evaluation sets, and CI fails if quality drops below a floor.

Q. Is 95.5% accuracy good?

It is good on my 44 synthetic cases, with a confidence interval of 88.6 to 100% because the set is small. It says nothing about real photos, and I always say that.

Q. Tell me about a bug you found.

The perception cache key used Python's hash(), which changes every time Python starts, so a shared cache could never hit. I switched to SHA-256 and added a test that fails on the old code.

Q. What do you do when CI fails?

Read which job failed. A test or gate failure means my change broke something, so I fix the code, not the test. Once a dependency audit failed on a CVE, and I upgraded the package.

G. Tricky questions**Q. Is this really "multi-agent"?**

It is a pipeline of bounded components, one of which is an LLM. There is no autonomous agent loop or tool calling — on purpose, because decisions must be explainable.

Q. Does RAG decide the claims?

No. RAG is only for reviewers' policy questions. Verdicts come from rules.

Q. What does a claim cost?

Billed cost is \$0 on free tiers. Each call's list-price cost is recorded, but I have not measured a real per-claim figure from live traffic yet, so I won't quote one.

Q. What would you improve first?

Real claim photos to recalibrate the quality gate, Postgres and a job queue for scale, and an entailment check that the copilot's wording matches the clause it cites.

14. Weaknesses to admit, and what comes next

Saying these first shows good engineering judgement. Being caught on them looks like overclaiming.

Weakness	What to say
The 44 test claims are synthetic	"Accuracy is on generated images; real photos need recalibration."
Golden questions and policy share an author	"The RAG eval is in-distribution; a larger external set is next."
Copilot wording not checked against the clause	"Citations are validated; wording is not — an entailment check is next."
One worker, in-process limits	"Rate limits and the job pool are correct for one process; Redis is the fix."
Free-tier storage is wiped on redeploy	"SQLite on an ephemeral disk; Postgres and object storage are the upgrade."
Refresh token in localStorage	"Different sites block cookies; same-domain httpOnly cookie is the upgrade."
No real per-claim cost yet	"Telemetry is live and will measure it; I don't quote a number I haven't measured."

What comes next (Phase B)

- Postgres (Neon or Supabase) with Alembic migrations, and object storage for photos.
- A Redis token bucket shared by all workers, and a real job queue (ARQ).
- A bounded, read-only Investigator Assistant for reviewers: a few read-only tools, at most 6 steps, a token budget, and tests proving customer text cannot trigger tools.
- A reviewer feedback loop: measure how often reviewers agree with AURELIX and tune thresholds on stored data.

15. 2-minute demo script

Step 1 — Claimant (30 s)

Open aurelix.space, click Explore as a claimant. Submit a car claim with one photo. Point at the 8-stage trace: "only the purple stage calls a model".

Step 2 — The verdict (20 s)

Show the verdict and rule id, then Policy basis: the clauses behind the rule. "No model involved — the customer's right to an explanation."

Step 3 — Reviewer (25 s)

Sign out, Explore as a reviewer. Show all claims and the review queue. "Roles are enforced on the server; another customer's claim is a 404."

Step 4 — Copilot (30 s)

Ask "Is rust covered?" — show the answer and EXC-2. Ask "What is the monthly premium?" — show "Not in the policy, no model was called".

Step 5 — Analytics and repo (15 s)

Show Model usage (calls, p50/p95, quota). Mention green CI and docs/SECURITY.md.

Last tip: if you don't know an answer, say what you do know and how you would find out. Honest and precise beats confident and wrong — that is also the whole philosophy of AURELIX.